Linked Lists

DSA27 — Lecture 05 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 5 · Fall 2026

Where We Are

Last week, one weakness

The dynamic array: $O(1)$ to index, amortised $O(1)$ to append —
and $O(n)$ to insert at the **front**, because **everything shifts**.

Today: a structure where nothing ever shifts, and nothing is ever copied.

Today

1. Nodes and references
2. Walking a list — and why indexing is $O(n)$
3. Inserting: at the front, in the middle, at the end
4. Removing — and who frees the node
5. Reversing in place, with three references
6. Array against linked list: costs, memory, measured
7. Variants: tail pointer, doubly linked, sentinel

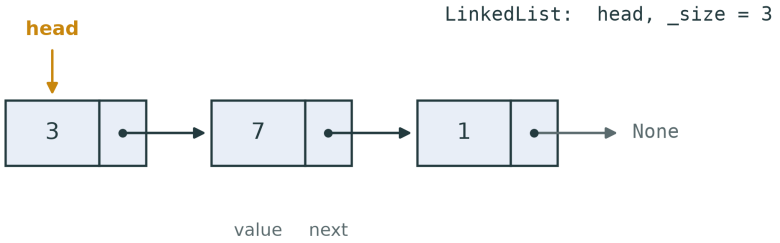
Nodes and References

A node: a value and a link

```
class Node:
    __slots__ = ("value", "next")

    def __init__(self, value, next=None):
        self.value = value
        self.next = next          # another Node, or None at the end
```

A linked list: nodes, each holding a value and a reference to the next



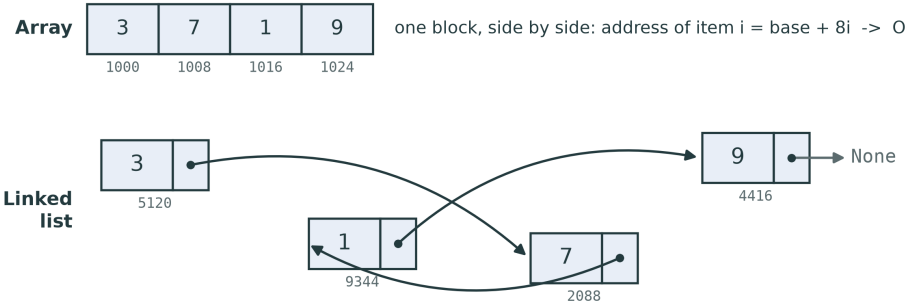
Build one by hand

```
>>> from dsa.linked_list import Node
>>> c = Node(1)
>>> b = Node(7, c)
>>> a = Node(3, b)
>>> a.value, a.next.value, a.next.next.value
(3, 7, 1)
>>> a.next.next.next is None
True
```

A reference **is** the arrow in the picture. `a.next` *is* `b`.

Two layouts, one difference

Same four values, two layouts — and the whole difference in cost



Walking the List

The traversal loop

```
node = self.head
while node is not None:
    # ... use node.value ...
    node = node.next
```

Every linked-list algorithm is this loop, plus a little bookkeeping.

- `node is not None` — stop **after** the last node
- `node = node.next` — the only way to move

Indexing is $O(n)$ — that is the lesson

	`DynamicArray[i]`	`LinkedList[i]`
How	base + i × slot size	follow `next` i times from head
Cost	$O(1)$	$O(i)$ — $O(n)$ worst case

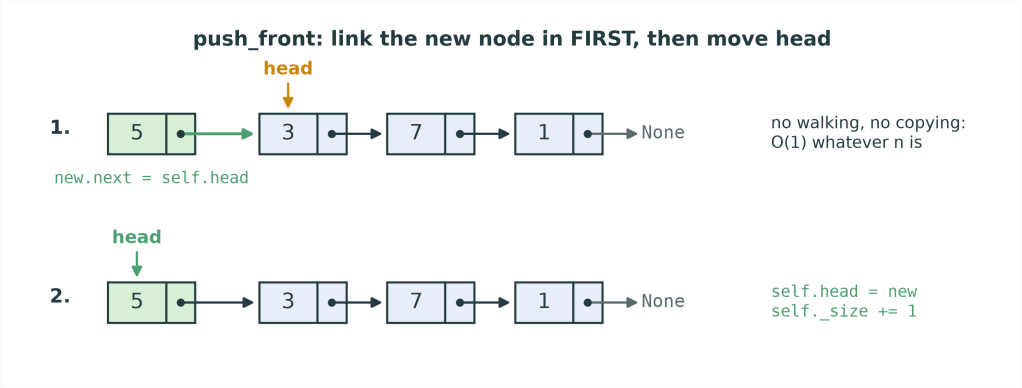
Same interface, `a[i]`. **Completely different cost.**

```
for i in range(len(ll)):
    print(ll[i])          # O(n) each time: O(n²) for the whole loop!
```

Iterate with `for value in ll:` — one walk, $O(n)$ in total.

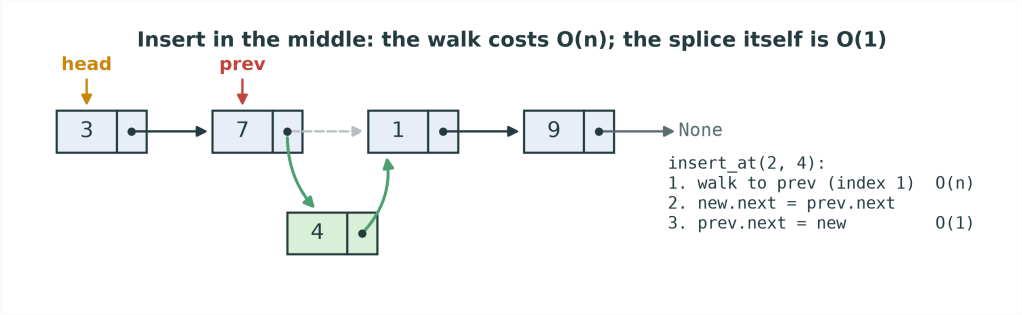
Inserting

At the front: $O(1)$




```
new = Node(value)
new.next = self.head      # 1. link the new node to the old first node
self.head = new           # 2. then make it the first node
```

In the middle: walk, then splice



To put a new node at index i, stop at the node **before** it (index i − 1), then:

```
new.next = prev.next
prev.next = new
```

At the end: $O(n)$ — unless you keep a tail

append must find the last node: walk from `head` until `node.next is None` — $O(n)$.

Keep a second reference, **`_tail`**, to the last node, and append becomes $O(1)$:

```
self._tail.next = new
self._tail = new
```

The price: **every** method that can change the last node must keep `_tail` correct.

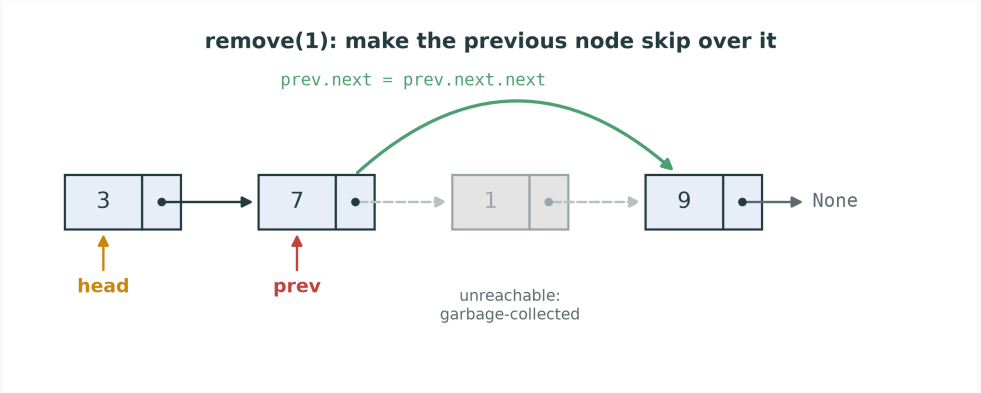
Removing

From the front: $O(1)$

```
if self.head is None:
    raise IndexError("pop from empty list")
value = self.head.value
self.head = self.head.next
self._size -= 1
return value
```

The old first node is no longer referenced by anything — so it is **gone**.

By value: skip over it

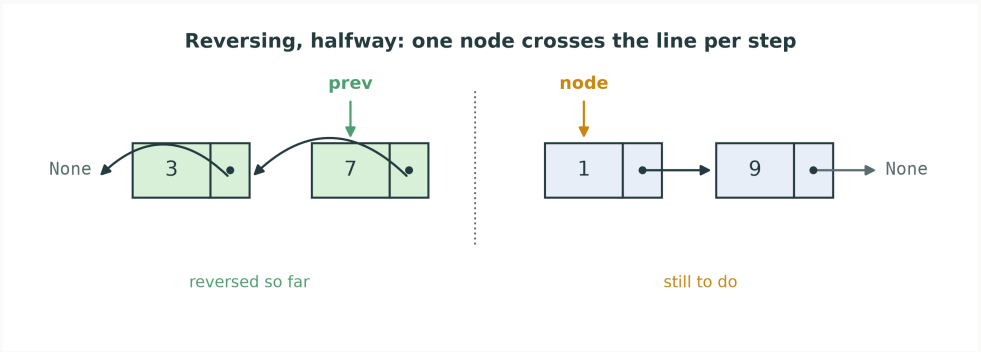


Find the node **before** the one to remove, then `prev.next = prev.next.next`.

The **head** is the special case: it has no `prev`.

Reversing in Place

Three references



Keep **prev** (the reversed part) and **node** (the rest). One step moves one node across the line:

1. remember `node.next` — you are about to overwrite it
2. point `node.next` back at `prev`
3. advance: `prev` becomes `node`, `node` becomes the remembered next

At the end, `prev` is the new head. $O(n)$ time, $O(1)$ extra space.

Array Against Linked List

The costs, side by side

Operation	`DynamicArray`	`LinkedList`
read / write position i	$O(1)$	$O(n)$
insert or remove at the **front**	$O(n)$	$O(1)$
append at the **end**	amortised $O(1)$	$O(n)$ — $O(1)$ with a tail
remove from the end	$O(1)$	$O(n)$ singly linked
insert / remove **given the position's neighbour**	$O(n)$	$O(1)$
search for a value	$O(n)$	$O(n)$
`len`	$O(1)$	$O(1)$ — because of `_size`
memory per element	one reference	a whole node

Measured, not asserted

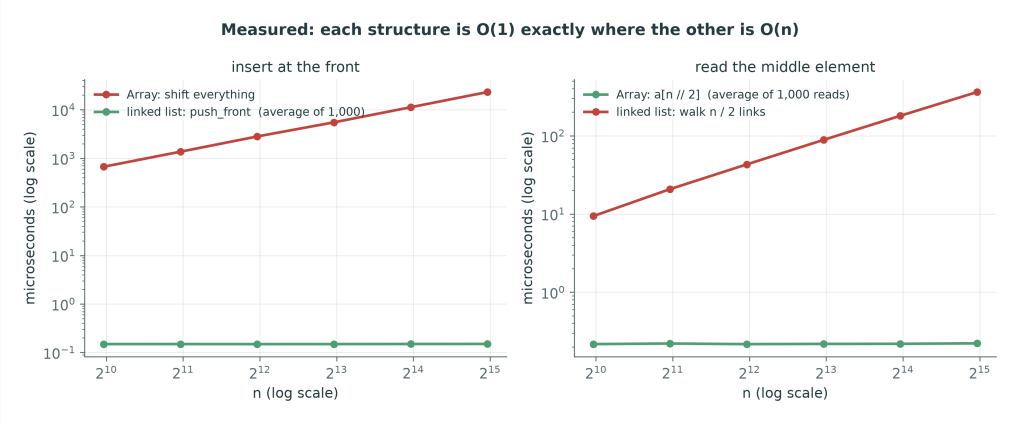


Figure: Real timings, reference implementations on the course `Array` and on nodes.

Memory: a node is not free

```
>>> import sys
>>> sys.getsizeof(Node(1))     # with __slots__, on 64-bit CPython 3.13
48
```

An array slot is **8 bytes** (one reference). A node is **48** — six times as much, per element, **plus** the value object both of them point to.

Variants

Doubly linked, and sentinels

Doubly linked: each node also holds `prev`.

- remove a node you hold in $O(1)$ — no need to find its predecessor
- walk both ways; remove from the end in $O(1)$
- costs one more reference per node, and twice the bookkeeping

Sentinel (dummy) node: a permanent, valueless node before the first real one.

- every real node now has a predecessor — **no special case for the head**

Circular: the last node points back to the first — useful for round-robin.

Watch it run: the Linked List Lab

Linked List Lab — open it in any browser.

- The code, the references and the **nodes and links**, one step at a time
- Insert at the front, in the middle, at the end; pop, remove, find, reverse
- **Doubly linked**: four links per insertion; pop from both ends in $O(1)$
- A tail pointer and a sentinel; counters of **hops** and **links changed**
- *What happens next?* — answer before you step
- Eight scenarios that **go wrong**: links in the wrong order, off by one, no head case, a forgotten `prev`…

This Week

Exercises: `dsa/linked_list.py`

Method	Target	The trap
`push_front(value)`	$O(1)$	link first, then move head
`append(value)`	$O(n)$ (tail: $O(1)$ challenge)	the empty list has no last node
`insert_at(index, value)`	$O(n)$	index 0 is `push_front`; stop at `index - 1`
`pop_front()`	$O(1)$	`IndexError` when empty
`remove(value)`	$O(n)$	the head has no `prev`; return `True`/`False`
`find(value)`	$O(n)$	`-1` when absent
`reverse()`	$O(n)$, $O(1)$ space	save `next` before you overwrite it
`__getitem__(index)`	$O(n)$	`IndexError` out of range

Every method that adds or removes a node: **update `_size`**.

```
pytest tests/test_linked_list.py -v
```

Homework 5 — before Lecture 06

1. **Implement** `dsa/linked_list.py` until `pytest tests/test_linked_list.py` passes.
2. **Tail pointer.** Add `_tail` and make append $O(1)$. List every method you had to change, and why.
3. **Measure.** Time `push_front` against `DynamicArray.insert_at(0, x)`, and `ll[n // 2]` against `arr[n // 2]`, for n = 1,000 … 32,000. Do your curves match the figure in this handout?
4. **On paper.** Trace `reverse()` on `[1, 2, 3, 4]`: draw `prev`, `node` and all the links after each step.

Summary

Seven things to keep

1. A linked list is **nodes** — a value and a `next` reference — plus a `head`.
2. Every algorithm is the **traversal loop**: `while node is not None: … node = node.next`.
3. **Indexing is** $O(n)$; iterate instead of indexing in a loop.
4. Front insert and remove: $O(1)$. Link the new node in **before** moving `head`.
5. Changes happen **from the node before**; the head is the special case (or use a sentinel).
6. Reverse with **three references**; save `next` first.
7. Array and linked list are **mirror images**: each is $O(1)$ where the other is $O(n)$ — choose by the operations you need.

Next

Week 6 — Stacks. One ADT, and the question this lecture prepares: array or linked list underneath? Then the applications that make stacks essential.